

**VISVESVARAYA TECHNOLOGICAL UNIVERSITY,
BELAGAVI - 590 018**



**Mini Project Report
On
SPACE MISSION ARCHIVE**

A report submitted in partial fulfilment of the requirements for

DATABASE MANAGEMENT SYSTEMS (BCS403)

in

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted by

J MANASA REDDY

4AL23AI022

M PREETY DEVI

4AL23AI066

Under the Guidance of

Mr. Lohith B

Assistant Professor



ALVA'S
Education Foundation®

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)**

ALVA'S INSTITUTE OF ENGINEERING AND TECHNOLOGY

MOODBIDRI-574225, KARNATAKA

2024 – 2025

ALVA'S INSTITUTE OF ENGINEERING AND TECHNOLOGY
MIJAR, MOODBIDRI D.K. -574225 KARNATAKA



**DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE
LEARNING**

CERTIFICATE

This is to certify that the Mini Project entitled **SPACE MISSION ARCHIVE** has been successfully completed by

J MANASA REDDY	4AL23AI022
M PREEY DEVI	4AL23A066

in the partial fulfilment for the award of Degree of Bachelor of Engineering in Computer and Engineering of the Visvesvaraya Technological University, Belagavi during the year 2024-2025. It is certified that all corrections/suggestions indicated have been incorporated in the report. The Mini project report has been approved as it satisfies the academic requirements in respect of Mini Project Work prescribed for the award of Bachelor of Engineering Degree.

Mr. LOHITH B

Prof. HARISH KUNDER

External Viva

Name of the Examiners

Signature with Date

1.

2.

ALVA'S INSTITUTE OF ENGINEERING AND TECHNOLOGY
MIJAR, MOODBIDRI D.K. -574225 KARNATAKA



**DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE
LEARNING**

DECLARATION

We,

J MANASA REDDY

M PREETY DEVI

Hereby declare that the dissertation entitled, **SPACE MISSION ARCHIVE** is completed and written by us under the supervision of our guide **Mr. Lohith B, Assistant Professor** Department of Computer Science and Engineering, Alva's Institute of Engineering and Technology of the Visvesvaraya Technological University, Belagavi during the year 2024- 2025. The dissertation report is original and it has not been submitted for any other degree in any university.

J MANASA REDDY 4AL23AI022

M PREETY DEVI 4AL23AI066

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany a successful completion of any task would be incomplete without the mention of people who made it possible, success is the epitome of hard work and perseverance, but steadfast of all is encouraging guidance.

So, with gratitude we acknowledge all those whose guidance and encouragement served as beacon of light and crowned the effort with success.

The selection of this mini project work as well as the timely completion is mainly due to the interest and persuasion of our mini project coordinator **Mr. Lohith B, Assistant Professor**, Department of Computer Science & Engineering. We will remember his contribution for ever.

We sincerely thank, **Mr Harish Kunder**, Professor and Head, Department of Artificial Intelligence and Machine Learning who has been the constant driving force behind the completion of the project.

We thank our beloved Principal **Dr. Peter Fernandes**, for his constant help and support throughout.

We are indebted to **Management of Alva's Institute of Engineering and Technology, Mijar, Moodbidri** for providing an environment which helped us in completing our mini project.

Also, we thank all the teaching and non-teaching staff of Department of Artificial Intelligence and Machine Learning for the help rendered.

J MANASA REDDY

4AL23AI022

M PREETY DEVI

4AL23AI066

ABSTRACT

This project involves the development of a comprehensive Flask-based web application designed to manage and present structured data related to space exploration. The application serves as a centralized platform for handling key entities such as space missions, space agencies, spacecrafts, and astronauts. Each of these entities is stored in its own database table and is fully integrated into the system with dedicated HTML templates and routes to support Create, Read, Update, and Delete (CRUD) operations.

The application includes a dynamic search and filtering system for improved data navigation—most notably, the astronauts module supports combined filters (e.g., by destination and name). A user authentication system has been implemented to secure access to the platform, requiring login credentials to reach the main dashboard, ensuring that data is protected and only accessible to authorized users.

Technologies used include Python (Flask framework) for backend logic, SQLite for the database layer, and HTML/CSS with JavaScript for the frontend. The architecture is modular and scalable, making it easy to expand the system to include additional entities or features in the future. This tool is particularly valuable for academic researchers, educators, and space enthusiasts who need an efficient way to organize, manage, and analyze data related to space missions and personnel.

TABLE OF CONTENTS

DECLARATION	3
ACKNOWLEDGEMENT	4
ABSTRACT.....	5
TABLE OF CONTENTS.....	6
LIST OF FIGURES	8
INTRODUCTION	9
1.1. INTRODUCTION ABOUT THE DATABASE.....	9
1.2. INTRODUCTION ABOUT THE PROJECT TOPIC	9
1.3. PROBLEM STATEMENT.....	10
1.4 . PROPOSED SOLUTIONS	10
SYSTEM REQUIREMENT SPECIFATION	11
2.1. Functional Requirements	11
2.2. Non-Functional Requirements	11
2.3. Hardware Requirements.....	11
2.4. Software Requirements	11
SYSTEM DESIGN	13
3.1. Database Design.....	13
3.2. Application Architecture	13
3.3. Workflow.....	13
3.4. ER DIAGRAM	14
3.5. SCHEMA DIAGRAM	15
IMPLEMENTATION	16
4.1 LANGUAGE USED FOR IMPLEMENTATION	16
4.2 PLATFORM USED FOR IMPLEMENTATION.....	16
4.3. SQL QUERY	17

RESULTS 19

CONCLUSION..... 21

 Key Benefits of the Project 21

REFERENCES 22

LIST OF FIGURES

Fig:3.4.1. ER Diagram of Space Station Database	14
Fig:3.5.1 The schema diagram for Space Mission Archive.	15
Fig:4.3.1 The Astronauts for the Query	18
Fig:4.3.2 The Astronauts for the Query	18
Fig:5.1.1. The Index page for Space Mission Archive.	19
Fig:5.1.2. The Missions Table for Space Mission Archive.	19
Fig:5.1.3. The Astronauts Table for Space Mission Archive.	20

INTRODUCTION

1.1. INTRODUCTION ABOUT THE DATABASE

A database is a structured collection of organized data that is stored and managed for efficient retrieval and manipulation. It serves as a centralized repository for storing and managing information in a systematic and structured manner. Databases play a crucial role in various applications and industries, providing a foundation for data storage, retrieval, and manipulation. Key components of a database include tables, which represent entities, and columns, which define the attributes or properties of those entities. Relationships between tables are established through keys, facilitating the organization of data and maintaining data integrity. The Database Management System (DBMS) is a software application that interacts with the database, enabling users to create, retrieve, update, and delete data. Databases are employed in a wide range of contexts, such as business, education, healthcare, finance, and more. They are instrumental in managing large volumes of information, ensuring data consistency, and supporting data-driven decision-making processes. Different types of databases exist, including relational databases, NoSQL databases, and in-memory databases, each designed to address specific requirements and preferences.

1.2. INTRODUCTION ABOUT THE PROJECT TOPIC

Overview

This project is a Flask-based web application developed to manage and organize data related to space exploration, including missions, agencies, spacecrafts, and astronauts. Each data category supports full CRUD (Create, Read, Update, Delete) operations and is presented through dedicated, user-friendly HTML templates. The application also features efficient search and filter functions, such as searching astronauts by name and filtering by destination. Built with Python, Flask, SQLite, HTML, and JavaScript, the system is modular, lightweight, and designed for easy future expansion.

Objectives

The primary objective of this project is to develop a web-based application using the Flask framework that efficiently manages and organizes data related to space exploration. The system aims to support full Create, Read, Update, and Delete (CRUD) operations across multiple interconnected entities, including missions, agencies, spacecrafts, and astronauts. Additionally, the application incorporates advanced search and filtering features to enable users to quickly find specific information within large datasets. Overall, the project is designed to be modular and scalable, ensuring ease of maintenance and the potential for future enhancements.

1.3. PROBLEM STATEMENT

Managing large and complex datasets related to space exploration—such as missions, agencies, spacecrafts, and astronauts—can be challenging due to the volume and interconnected nature of the information. Traditional methods, like manual record-keeping or fragmented tools, often lead to inefficiency, errors, and difficulty in accessing relevant data quickly. There is a need for a centralized, user-friendly system that enables easy management, updating, and retrieval of space mission data. This project addresses this problem by developing a web-based application using Flask that streamlines data organization, provides full CRUD functionality, and supports advanced search and filtering to improve data accessibility and usability.

1.4 . PROPOSED SOLUTIONS

To effectively overcome the difficulties associated with managing large and interconnected datasets in the domain of space exploration, this project proposes the development of a robust, web-based application built on the Flask framework using Python. The solution centers around creating a centralized data management system that organizes key entities—such as space missions, agencies, spacecrafts, and astronauts—into distinct yet related database tables. Each table supports full Create, Read, Update, and Delete (CRUD) operations, enabling users to maintain accurate, up-to-date records effortlessly.

The application’s architecture employs modular design principles by using dedicated HTML templates and backend routes for each entity, which improves maintainability and allows future expansion without disrupting existing functionality. To enhance usability, the system incorporates advanced search and filtering features, such as the ability to search astronauts by name and filter them by destination, helping users to quickly locate and analyze specific data within large datasets.

The backend utilizes SQLite as a lightweight and efficient database solution integrated seamlessly with Flask, providing a reliable storage mechanism without the need for complex setup. On the frontend, the application leverages HTML, CSS, and JavaScript to deliver a responsive and interactive user experience that facilitates smooth data entry, updates, and browsing.

By implementing this solution, the project aims to streamline the management and accessibility of space mission data, addressing the limitations of traditional manual or fragmented data handling methods. Ultimately, this solution seeks to empower researchers, educators, and space enthusiasts with a powerful and easy-to-use tool for organizing and exploring complex space exploration information efficiently.

SYSTEM REQUIREMENT SPECIFICATION

2.1. Functional Requirements

These are the core features the system must perform:

- Allow users to create, view, update, and delete records for missions, agencies, spacecrafts, and astronauts.
- Provide separate interfaces for each data category.
- Support search and filter options (e.g., search astronauts by name, filter by destination).
- Validate user input to ensure data accuracy.
- Use SQLite database to store and manage data with relational integrity.
- Ensure the web interface is responsive and user-friendly across devices.

2.2. Non-Functional Requirements

These requirements define how the system should behave:

- Usability: The web interface should be simple, readable, and user-friendly.
- Performance: The system should execute SQL queries and render results efficiently with minimal delay.
- Scalability: The design should allow for additional features like CRUD operations, search, or visual dashboards.
- Portability: The system should run on any standard Windows or Linux environment with MySQL and Python installed.
- Maintainability: Code and database schema should be modular and easy to update or expand.

2.3. Hardware Requirements

- Processor: Intel i3 or higher
- RAM: Minimum 4 GB
- Storage: At least 500 MB free disk space for project files and MySQL installation
- Internet: Optional (for installing dependencies and future hosting)

2.4. Software Requirements

- Operating System: Windows 10/11 or Linux
- Database: MySQL Server & MySQL Workbench

- Programming Language: Python 3.8 or above
- Libraries:
 - Flask
 - mysql-connector-python
 - Jinja2
- Code Editor: VS Code or any Python IDE
- Web Browser: Chrome, Firefox, or Edge (for viewing Flask web pages)

SYSTEM DESIGN

3.1. Database Design

The database is designed to organize space exploration data into four main tables, each representing a key entity:

- Missions: Stores information about space missions, including mission name, launch date, destination, and associated spacecraft.
- Agencies: Contains details about space agencies involved in missions, such as agency name, country, and type.
- Spacecrafts: Records data on spacecrafts used in missions, including spacecraft name, type, and manufacturer.
- Astronauts: Holds information on astronauts, including their name, nationality, missions participated in, and destination.

3.2. Application Architecture

The application is designed following a client-server architecture:

- Frontend Layer: HTML, CSS, and JavaScript provide the user interface in the web browser.
- Backend Layer: Flask (Python) handles application logic, processes user requests, and communicates with the database.
- Database Layer: SQLite stores and manages all data related to missions, agencies, spacecrafts, and astronauts.

3.3. Workflow

- User sends a request to the Flask web server (e.g., accessing a webpage).
- Flask routes the request to the appropriate function.
- The backend connects to the MySQL database and executes SQL queries to retrieve or update data.
- Data retrieved from the database is passed to HTML templates.
- The browser renders the HTML page with the fetched data for user interaction.

3.4. ER DIAGRAM

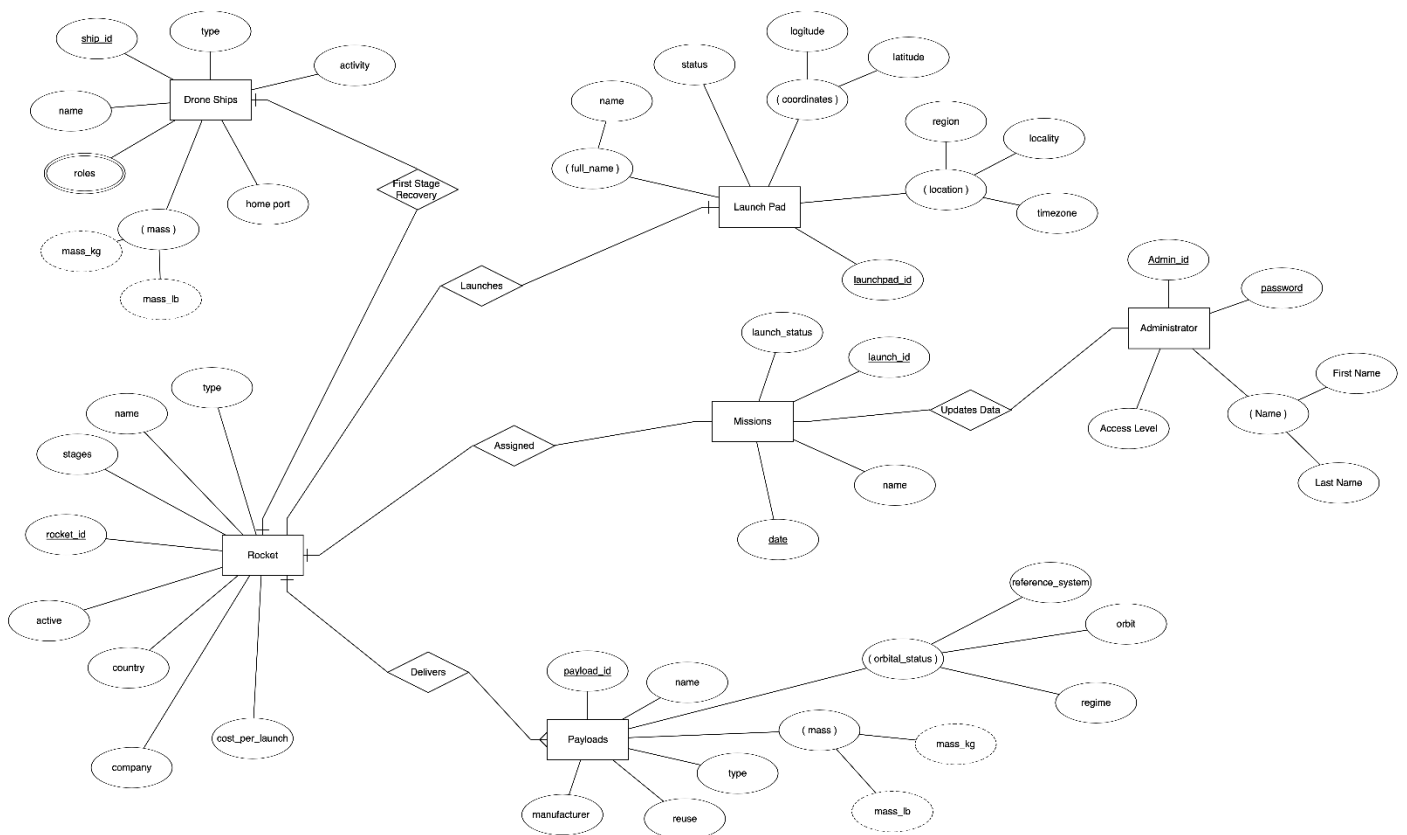


Fig:3.4.1. ER Diagram of Space Station Database

3.5. SCHEMA DIAGRAM

The relational database schema defines the structure of the data by specifying the tables, columns, and data types. It also defines the relationships between the tables by specifying the primary key and foreign key constraints. A primary key is a unique identifier for each row in a table, while a foreign key is a reference to the primary key of another table.

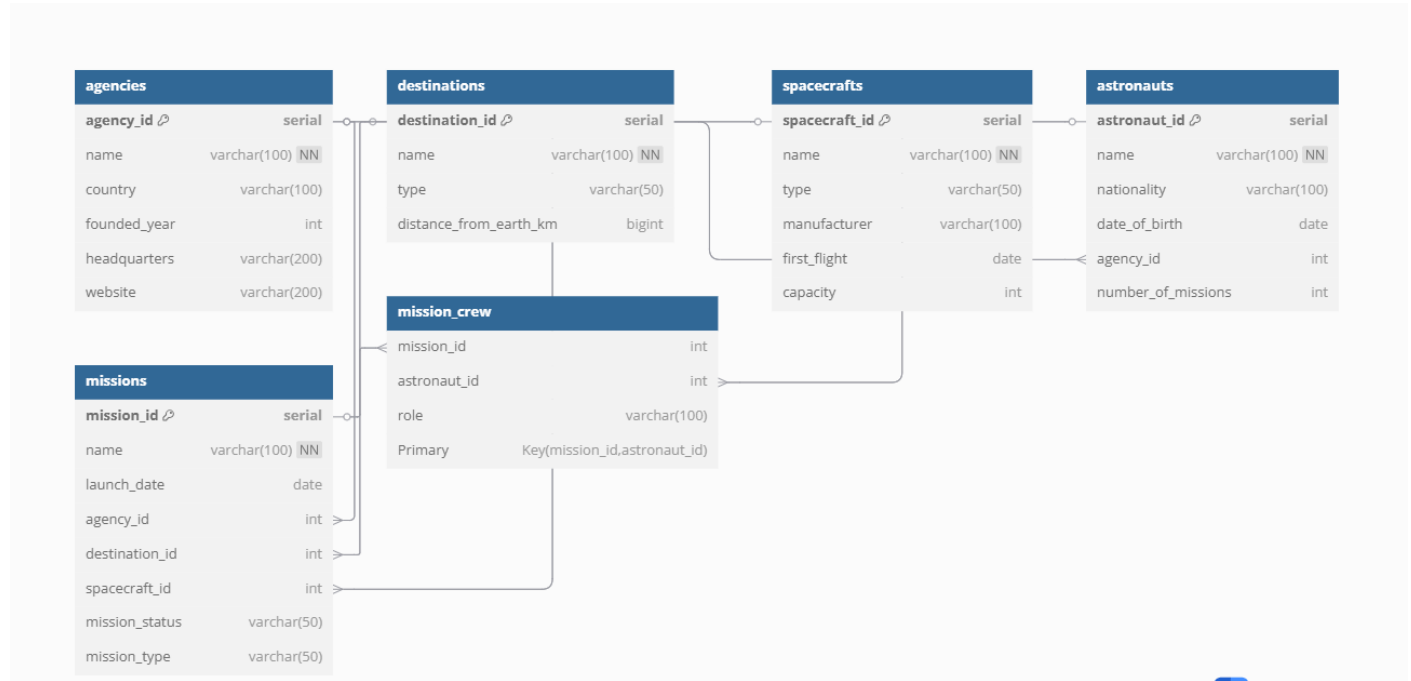


Fig:3.5.1 The schema diagram for Space Mission Archive.

IMPLEMENTATION

4.1 LANGUAGE USED FOR IMPLEMENTATION

The project was implemented using Python for backend development with the Flask framework, allowing smooth interaction with the MySQL database using SQL queries. HTML, JavaScript and CSS were used to design the frontend interface, while Jinja2 templating enabled dynamic rendering of data on web pages. This combination of languages ensured a functional and user-friendly DBMS web application.

4.1.1. Python

- Purpose: Used for backend development.
- Reason: Python is a high-level, easy-to-learn language with strong support for database interaction, making it suitable for server-side development using Flask.

4.1.2. SQL (Structured Query Language)

- Purpose: Used to create and manage the database in MySQL.
- Reason: SQL is essential for performing operations such as creating tables, inserting records, retrieving data, and executing queries.

4.1.3. HTML (HyperText Markup Language)

- Purpose: Used to create the structure of web pages.
- Reason: HTML is the standard language for web page design and is used to display data fetched from the backend.

4.1.4. CSS (Cascading Style Sheets)

- Purpose: Used for styling HTML pages.
- Reason: CSS helps in making the frontend user interface visually appealing and responsive.

4.2 PLATFORM USED FOR IMPLEMENTATION

4.2.1. MySQL Workbench

MySQL Workbench is a visual database design and administration tool used to create, manage, and maintain MySQL databases. In this project, it was used to:

- Design the database schema by defining tables, their fields, and relationships.

- Write and execute SQL scripts to create tables (CREATE TABLE), insert data (INSERT INTO), and manipulate data.
- Test queries to ensure that the data retrieval and relationships between tables are functioning correctly.
- Manage the database efficiently, ensuring data integrity and normalization.

4.2.2. Visual Studio Code (VS Code)

VS Code is a popular, free, and open-source code editor with features like syntax highlighting, debugging, and extensions support. For this project, VS Code:

- Provides an organized workspace to manage multiple files such as SQL scripts, Python backend code, HTML templates, and CSS stylesheets.
- Helps in running Python scripts, viewing outputs, and debugging errors.
- Supports version control integration (Git), facilitating better project management.

4.2.3. Web Browser (Google Chrome)

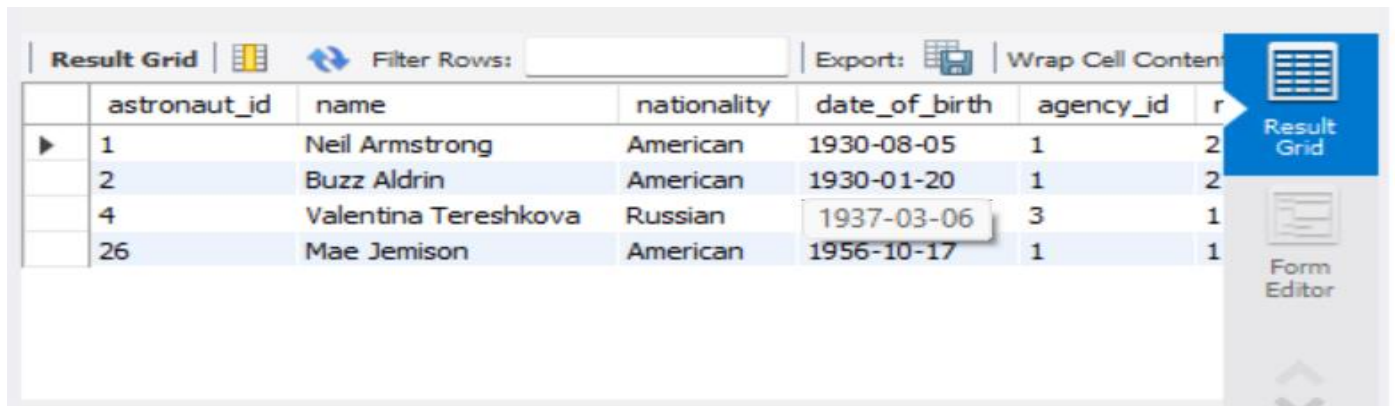
A web browser is used to access the web application developed with Flask. It sends requests to the Flask server and displays the dynamically generated web pages to users. This allows users to:

- View mission details, rockets, launches, and other data interactively.
- Navigate the database through a simple, intuitive graphical interface without needing to know SQL commands.
- Test the application's functionality during development.

4.3. SQL QUERY

```
SELECT DISTINCT a.*
FROM astronauts a
JOIN mission_crew ma ON a.astronaut_id = ma.astronaut_id
JOIN missions m ON ma.mission_id = m.mission_id
JOIN destinations d ON m.destination_id = d.destination_id
```

WHERE d.name = "Moon";



The screenshot shows a 'Result Grid' window with a table of astronaut data. The table has columns: astronaut_id, name, nationality, date_of_birth, agency_id, and a row number column. The data is as follows:

astronaut_id	name	nationality	date_of_birth	agency_id	
1	Neil Armstrong	American	1930-08-05	1	2
2	Buzz Aldrin	American	1930-01-20	1	2
4	Valentina Tereshkova	Russian	1937-03-06	3	1
26	Mae Jemison	American	1956-10-17	1	1

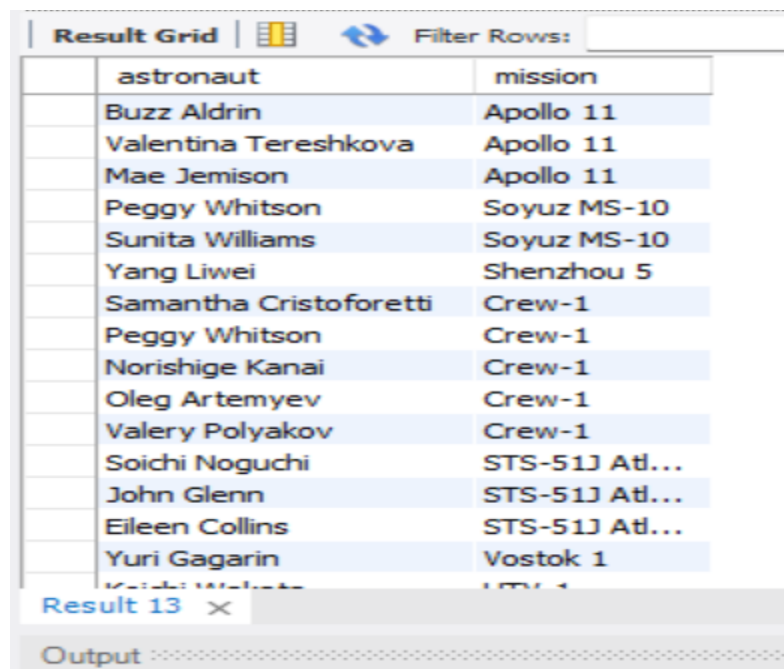
Fig:4.3.1 The Astronauts for the Query

SELECT a.name AS astronaut, m.name AS mission

FROM astronauts a

JOIN mission_crew ma ON a.astronaut_id = ma.astronaut_id

JOIN missions m ON ma.mission_id = m.mission_id;



The screenshot shows a 'Result Grid' window with a table of astronaut and mission data. The table has columns: astronaut and mission. The data is as follows:

astronaut	mission
Buzz Aldrin	Apollo 11
Valentina Tereshkova	Apollo 11
Mae Jemison	Apollo 11
Peggy Whitson	Soyuz MS-10
Sunita Williams	Soyuz MS-10
Yang Liwei	Shenzhou 5
Samantha Cristoforetti	Crew-1
Peggy Whitson	Crew-1
Norishige Kanai	Crew-1
Oleg Artemyev	Crew-1
Valery Polyakov	Crew-1
Soichi Noguchi	STS-51J Atl...
John Glenn	STS-51J Atl...
Eileen Collins	STS-51J Atl...
Yuri Gagarin	Vostok 1

Fig:4.3.2 The Astronauts for the Query

RESULTS



Fig:5.1.1. The Index page for Space Mission Archive.

Missions Dashboard

Search by name...

Mission Name: dd-mm-yyyy Agency ID: Destination ID: Spacecraft ID: Status (e.g. success) Type (e.g. manned) Save

ID	Name	Launch Date	Agency	Destination	Spacecraft	Status	Type	Actions
1	Apollo 11	1969-07-16	1	2	1	success	manned	 
2	Soyuz MS-10	2018-10-11	3	4	2	ongoing	manned	 
3	Shenzhou 5	2003-10-15	6	1	3	success	manned	 
4	Crew-1	2020-11-16	1	4	4	ongoing	manned	 
5	SpaceX Starship Test	2023-04-20	7	25	5	success	unmanned	 
6	STS-51J Atlantis	1985-10-03	1	4	6	success	manned	 

Fig:5.1.2. The Missions Table for Space Mission Archive.

Astronauts Dashboard

Search by Name:
Filter by Destination:

All Destinations

Add / Update Astronaut

Name

Nationality

dd-mm-yyyy

Agency ID

Missions

Save

ID	Name	Nationality	Date of Birth	Agency ID	Number of Missions	Actions
1	Neil Armstrong	American	1930-08-05	1	2	
2	Buzz Aldrin	American	1930-01-20	1	2	
3	Yuri Gagarin	Russian	1934-03-09	3	1	

Fig:5.1.3. The Astronauts Table for Space Mission Archive.

Mission Crew Dashboard

Search by role...

Mission ID
Astronaut ID
Role

Save

Reset

Mission ID	Astronaut ID	Role	Actions
1	1	Commander	
1	2	Lunar Module Pilot	
1	4	Module Pilot	
1	26	Mission Specialist	
2	7	Commander	
2	22	Flight Engineer	
3	9	Commander	
4	6	Mission Specialist	

Fig:5.1.4. The Mission Crew Table for Space Mission Archive.

CONCLUSION

In a Flask-based web application, different programming languages work together to deliver a complete and user-friendly experience. Python is used on the backend to handle server-side logic, process data, interact with databases, and respond to user requests. It forms the core of the application's functionality and decision-making.

On the frontend, HTML structures the content that users see in their browsers, defining elements like tables, forms, and buttons. CSS is then used to style this content, ensuring the design is clean, attractive, and responsive across devices. To make the application interactive, JavaScript adds dynamic behavior—such as instant search, form validation, and responsive updates—without needing to reload the page.

Together, Python, HTML, CSS, and JavaScript create a well-rounded web application that is functional, visually appealing, and interactive for users.

Key Benefits of the Project

1. **Centralized Data Management:** The project provides a single platform to manage and organize data related to space missions, agencies, spacecrafts, and astronauts. This centralization improves accessibility and reduces data duplication or inconsistency.
2. **User-Friendly Interface:** With separate HTML templates and a clean design, users can easily navigate, view, search, and update information. CSS and JavaScript ensure the interface is both attractive and interactive.
3. **Full CRUD Functionality:** The system allows users to Create, Read, Update, and Delete records for all entities. This makes it easy to keep the data up to date without needing technical knowledge or database access.
4. **Search and Filter Capabilities:** Built-in search and filtering features (e.g., searching astronauts by name or filtering by destination) enhance usability by helping users quickly find specific information.
5. **Modular and Scalable Design:** Each table (missions, agencies, spacecrafts, etc.) is handled with separate routes and templates, making the codebase modular and easy to expand as new data types or features are added.
6. **Secure Access:** With the addition of a login system, only authorized users can access or modify sensitive data. This helps protect the integrity and confidentiality of mission information.
7. **Educational Value:** The project is an excellent learning tool for understanding web development using Flask. It combines backend logic, frontend design, form handling, data validation, and session management in a real-world context.

REFERENCES

1. Books & Academic Resources:

- Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson Education.

2. Web Resources:

- Mysql Documentation– Official documentation for MySQL database system.
- Flask Documentation – Official documentation for Flask web framework.

3. Open-Source GitHub Repositories Studied:

- <https://github.com/r0han99/spaceX-database-system>